Génération de trajectoires dans l'espace de la tâche avec un robot manipulateur

Objectif

Le but de ce bureaux d'études (BE) est de développer une primitive de mouvement afin de faire réaliser à l'organe terminal d'un robot manipulateur un mouvement avec des contraintes en position, en vitesse ou en effort.
La trajectoire dans l'espace opérationnel se fait en respectant les contraintes de la tâche. Les modèles inverses du robot manipulateur permettent de calculer les commandes articulaires (position, vitesse, accélération ou couple) à envoyer au robot pour générer le mouvement désiré.
La tâche est donc définie dans l'espace opérationnel et il est demandé de mettre en évidence le lien entre l'espace opérationnel et l'espace généralisé.
Le travail doit être fait de manière **générique** afin de pouvoir facilement modifier la trajectoire de la tâche (ou le robot).

Travail à réaliser :

- calculer les différents modèles,
- calculer les trajectoires $\boldsymbol{X}(\mathrm{t})$, $\dot{\boldsymbol{X}}(\mathrm{t})$et $\ddot{\boldsymbol{X}}(\mathrm{t})$ à chaque instants d'échantillonnage,
- calculer les trajectoires $\boldsymbol{q}(\mathrm{t})$, $\dot{\boldsymbol{q}}(\mathrm{t})$, $\ddot{\boldsymbol{q}}(\mathrm{t})$ à chaque instants d'échantillonnage,
- simuler vos trajectoires.

Temps estimé : 12 h en présence des encadrants (en 4 séances de 3h, 3h30, 2h, 3h30) et 4h-12h hors encadrement.

Exemple d'organisation temporelle :

- séance 1 : calcul des modèles directs (géométrique III et cinématique IV),
- séance 2 : prise en main du simulateur et calcul de la trajectoire opérationnelle,
- séance 3 : simulation de la trajectoire opérationnelle et affichage des courbes demandées,
- séance 4 : simuler la réalisation de la tâche.

Evaluation : chaque équipe doit se répartir le travail afin de pouvoir le réaliser dans le temps imparti. La note est basée sur plusieurs évaluations

- des contrôles oraux sur le travail réalisé lors des séances,
- une évaluation orale terminale individuelle lors de la séance d'évaluation,
- la synthèse du travail réalisé sur un court rapport,
- l'évaluation de votre code et des tests après la livraison finale.

Le sujet sera complété et précisé à chaque séance en fonction de sa finalité.

I Le robot TX-60l

Le robot TX-60l est un robot 6R de la gamme STAUBLI. La figure 1 vous indique ses axes de rotations ainsi que ses dimensions nécessaires pour établir les modèles.

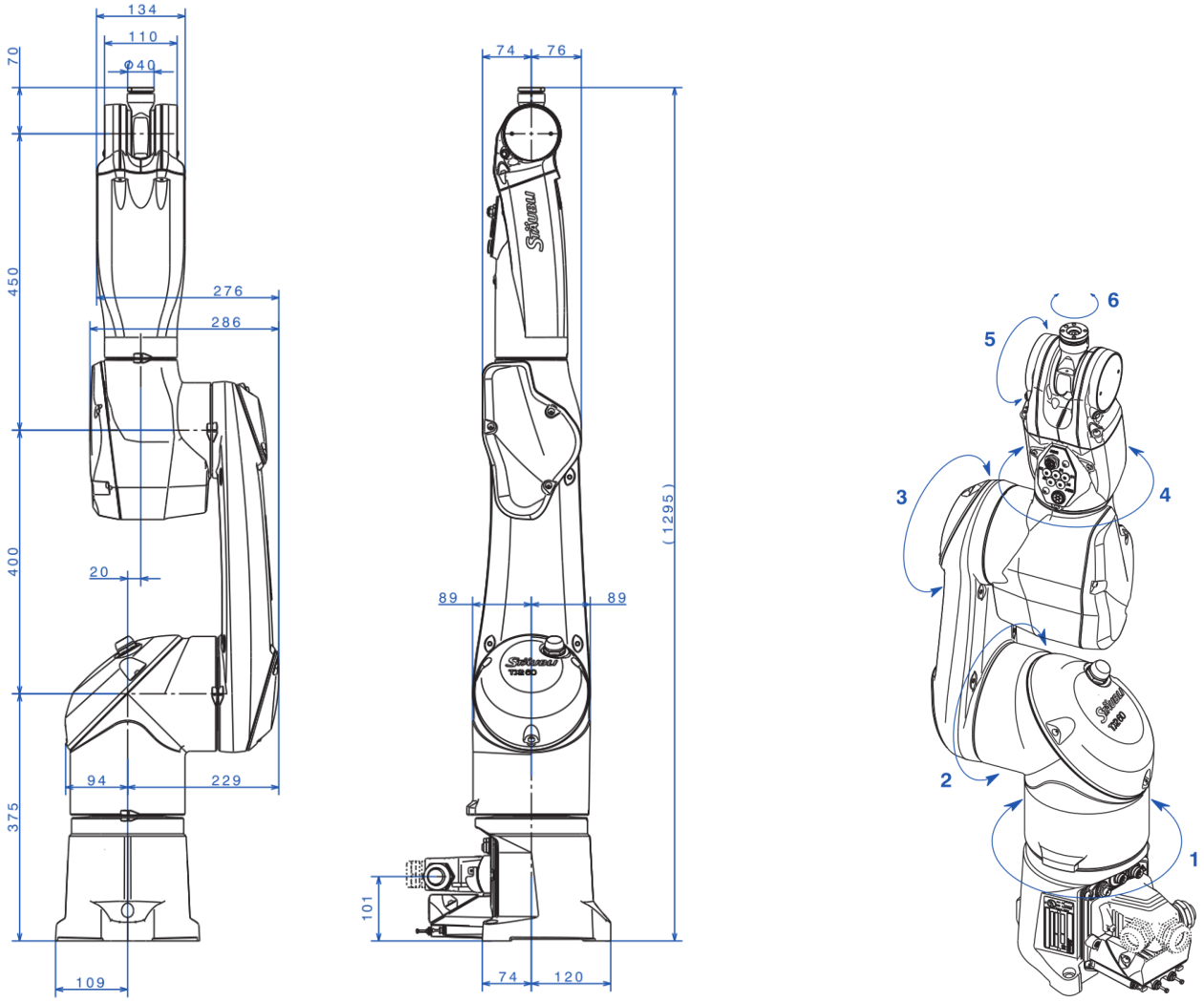


FIGURE 1 – Dimensions géométriques de l'UR3

La configuration du robot est définie dans l'espace généralisé par $\boldsymbol{q} = (q_1, q_2, q_3, q_4, , q_5, q_6)^T \in \mathbb{R}^6$.
Les butées mécaniques, les vitesses et les accélérations maximales du robot sont à lire dans le fichier *urdf* du robot.
La situation de l'O.T. est définie par la position du point O_e et par l'orientation du repère de l'effecteur R_e par rapport au repère de base R_0, (O_0, x_0, y_0, z_0).Ces deux repères sont fixés et ils correspondent aux repères du robot dans le simulateur (**ils ne doivent pas être modifiés !**).

II Simulation du robot TX-60l

Il faut simuler la boucle générale de contrôle du robot (figure 2) à l'aide d'un simulateur qui doit intégrer tous les effets dynamiques afin de se comporter "quasiment" comme le vrai robot dans son environnement réel.

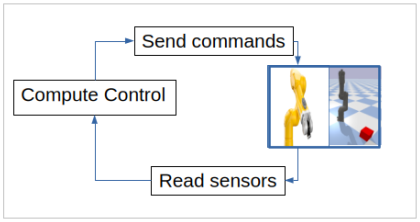


FIGURE 2 – Boucle générale de contrôle

A chaque itération (figure 3) le simulateur exécute le cycle suivant :

- Lecture des commandes : réception des commandes du contrôleur (position, vitesse, couple),
- Détection et gestion des collisions : calcul des intersections des objets géométriques,
- Calcul de la dynamique : intégration numérique des équations du mouvement (souvent la méthode d'Euler semi-implicite ou de Runge-Kutta) pour passer de l'état x_k à l'état x_{k+1} sur la durée de Δt
- Génération de capteurs virtuels : mise à jour des capteurs virtuels (coordonnées articulaires, calcul de la nouvelle image de la caméra...).
- Rendu graphique : rafraichissement de la vue 3D/GUI.

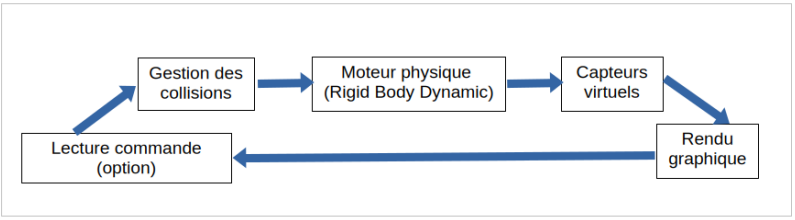


FIGURE 3 – Boucle de simulation avec un pas de temps de Δt

Afin de simuler l'exécution de vos consignes, nous utiliserons le simulateur PyBullet.
PyBullet est un simulateur de physique open source très répandu en robotique, reconnu pour sa précision, sa simplicité d'utilisation et sa rapidité. Basé sur la bibliothèque Bullet Physics, il est conçu pour simuler des interactions physiques de manière réaliste, ce qui en fait un excellent choix pour concevoir, tester et affiner des systèmes robotiques dans un environnement virtuel.

Après avoir décompressé l'archive disponible sous moodle vous récupérerez les fichiers suivants :

- un fichier de type notebook *introduction_pybulletTX60l.ipynb* qui montre l'utilisation principale de PyBullet,
- un répertoire *staubli/staubli_tx60_support/* qui contient l'ensemble des répertoires et fichiers nécessaires à la simulation :
  - dans *urdf* se trouve le fichier *tx60l.urdf* pour la modélisation du robot (position des repères, longueurs, masses, inerties,...des différents corps et liaisons),
  - dans *meshes* les répertoires *collision* et *visual* pour le calcul des collisions et du rendu visuel.

  Ne pas modifier ces fichiers.

TRAVAIL A REALISER

III Modélisation géométrique du robot TX-60l

Modélisation géométrique directe - MGD

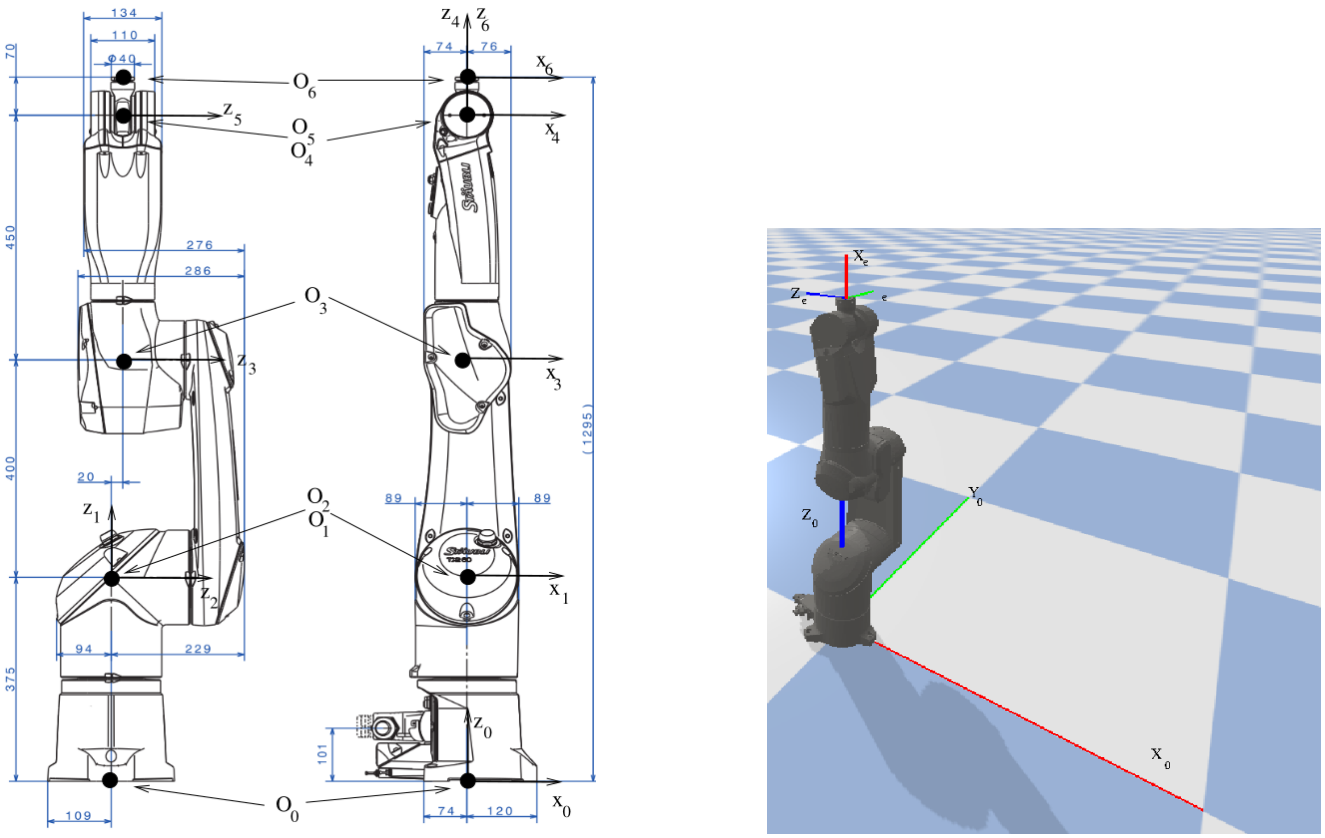


FIGURE 4 – Positionnement des repères pour $\boldsymbol{q} = (0, 0, 0, 0, 0, 0)^T$

- Calculer les changements de repère relatif entre chaque liaison.
- Calculer le MGD pour une situation $\boldsymbol{X}$ donnée par la position cartésienne de l'O.T. et par l'orientation de R_e en angle d'euler.
- Programmer une fonction Python qui calcule votre MGD de manière numérique.
- Spécifier les tests effectués pour vérifier votre modèle.
- Comparer votre MGD avec le MGD fournie par le simulateur : précision ? temps CPU ? Conclusion.

Modélisation géométrique inverse

La fonction *calculateInverseKinematics* du simulateur (voir documentation PyBullet) fait le calcul numérique du MGI.

- Spécifier les tests effectués pour vérifier que l'utilisation de votre fonction MGD et MGI soit cohérente.

IV Modélisation différentielle du robot TX-60l

Modélisation différentielle directe - MDD

- Calculer la jacobienne (géométrique) du robot.
- Calculer la jacobienne analytique permettant de connaître $\dot{\boldsymbol{X}}_{\boldsymbol{e}} = (\dot{\boldsymbol{x}}_e, \dot{\boldsymbol{y}}_e, \dot{\boldsymbol{z}}_e)^T$.
- Implémenter en Python votre fonction MDD.
- Tester vos fonctions jacobiennes. Préciser les cas de test.
- Comparer votre jacobienne avec la jacobienne fournie par le simulateur : précision ? temps CPU ? Conclusion.

Modélisation différentielle inverse

Ce calcul sera fait de manière numérique en utilisant directement le calcul de votre jacobienne et son inverse. (séance 1 et 2).

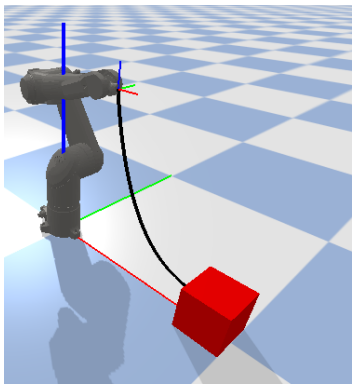
- Spécifier les tests effectués pour vérifier que l'utilisation de votre fonction MDD et MDI soit cohérente.

V Génération de mouvement

Cahier des charges

Développer une primitive de mouvement pour que l'OT suive une trajectoire $\boldsymbol{X}$(t)=$(x(t), y(t), z(t))^T$ de type polynomiale d'un point A à un point B avec les contraites suivantes :

- la trajectoire est définit dans un plan perpendiculaire au sol,
- la vitesse en A est définie par V_A et en B par V_B,
- l'accélération en A est définie par K_A et en B par K_B,
- l'orientation de l'axe X_e de l'OT reste constante, dans le plan de la trajectoire et parallèle au plan (x, y).



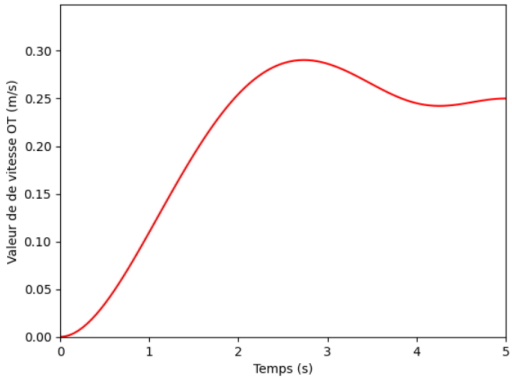


FIGURE 5 – Trajectoire de la tâche et profil de vitesse de l'abscisse curviligne imposé.

L'opérateur rentre :

- les coordonnées de A et de B,
- les vitesses V_A et V_B,
- les accélérations K_A et K_B.

La primitive $traj(A, B, V_A, V_B, K_A, K_B)$ retourne les vecteurs $\boldsymbol{X}$(t), $\dot{\boldsymbol{X}}$(t), $\ddot{\boldsymbol{X}}$(t).

Sachant que vous devez envoyer les consignes au robot avec une période Te (de l'ordre de 1 à 10 ms) vous devez calculer les différentes valeurs à chaque instants d'échantillonnage.

V.1 Loi de mouvement temporelle

V.2 Trajectoire opérationnelle polynomiale

- Calculer les coordonnées ($x(t)$, $y(t)$, $z(t)$) de la trajectoire en fonction des entrées de consigne.
- Calculer les vitesses et les accélérations opérationnelles ($\dot{x}(t)$, $\dot{y}(t)$, $\dot{z}(t)$), ($\ddot{x}(t)$, $\ddot{y}(t)$, $\ddot{z}(t)$).
- Calculer la vitesse et l'accélération du point O_E de l'OT.
- Afficher l'ensemble de ces courbes et vérifier que vos résultats sont cohérents ainsi que la trajectoire opérationnelle en 3D.
- Prorammer le calcul de la trajectoire opérationnelle.

V.3 Génération de mouvement dans l'espace articulaire (généralisé)

Cette primitive qui doit retourner les vecteurs $\boldsymbol{q}$(t) , $\dot{\boldsymbol{q}}$(t) et $\ddot{\boldsymbol{q}}$(t) .x

- A l'aide des modèles inverses, calculer et programmer $\boldsymbol{q}$(t) , $\dot{\boldsymbol{q}}$(t) , $\ddot{\boldsymbol{q}}$(t) (entrées de la commande)
- Afficher l'ensemble de ces courbes et vérifier que vos résultats sont cohérents avec la trajectoire opérationnelle en 3D.

VI Simulation dynamique (séance 3 et 4)

A VENIR